

EV Battery Monitoring System

Distributed 4-ECU SocketCAN Design, Implementation & Test Report

SocketCAN Assignment — Option 5

Name: Karan

Course: Automotive Communication Networks

Repository: [kAPEXLab/automotive-communication-networks — assignments/SocketCAN2.md](#)

Contents

1. Problem Description & Architecture
2. CAN Matrix & Signal Specification
3. Design Decisions & Fault Handling Strategy
4. Demonstration — Challenge 1: Normal Operation
5. Demonstration — Challenge 2: Missing Message Detection
6. Demonstration — Challenge 3: Sensor Fault Injection (Temp + Voltage)
7. Demonstration — Challenge 4: Node Failure (Controller + Actuator)
8. Demonstration — Challenge 5: Recovery
9. Test Case Results
10. Conclusion & Known Limitations

1. Problem Description & Architecture

An Electric Vehicle Battery Management System (BMS) must continuously monitor pack health (cell voltage, temperature, current), estimate State of Charge (SOC), and protect the pack by controlling the main contactor (isolating the pack on fault) and the cooling fan (preventing thermal runaway). This project implements a distributed 4-ECU simulation over SocketCAN on a virtual CAN bus (vcan0), plus a Diagnostic ECU providing bus-level fault detection independent of the other nodes.

```
[Voltage Sensor] [Temp Sensor] [Current Sensor]
  \           |           /
    SENSOR ECU (0x100/0x101/0x102/0x1F0)
    CONTROLLER ECU - computes SOC, decides mode
      /(0x110) \/(0x111)
[Contactor] [Cooling Fan] ←via ACTUATOR ECU
```

All messages also observed by: **DIAGNOSTIC ECU** → 0x1FF → **HMI**

All 4 ECUs plus the HMI attach to a single shared vcan0 bus, each as an independent OS process — analogous to independent physical ECUs on a vehicle CAN bus.

ECU	Role	Inputs	Outputs
Sensor ECU	Simulated sensing + self-check	(simulated values)	0x100, 0x101, 0x102, 0x1F0
Controller ECU (BMS)	SOC estimation, mode decision, safety logic	0x100, 0x101, 0x102, 0x1F0	0x110, 0x111, 0x120
Actuator ECU	Executes contactor/fan commands	0x110, 0x111	(physical action, logged)
Diagnostic ECU	Bus-wide timeout & plausibility watchdog	all messages	0x1FF

2. CAN Matrix & Signal Specification

ID	Name	Tx	Rx	Type	Rate
0x100	Cell Voltage	Sensor	Ctrl, Diag	Periodic	100 ms
0x101	Cell Temperature	Sensor	Ctrl, Diag	Periodic	200 ms
0x102	Pack Current	Sensor	Ctrl, Diag	Periodic	100 ms
0x120	BMS Status (SOC+Mode)	Controller	Diag, HMI	Periodic	500 ms
0x110	Contactor Command	Controller	Actuator	Event	on change
0x111	Fan Command	Controller	Actuator	Event	on threshold
0x1F0	Sensor Fault Flags	Sensor	Ctrl, Diag	Event	on fault
0x1FF	System Warning	Diagnostic	All	Event	on fault/timeout

Signal	Bytes	Res.	Range	Formula
Cell Voltage	2 uint16 LE	0.001V	0-5V	raw=Vx1000
Cell Temp	1 uint8	1C	-40 to 120C	raw=T+40
Pack Current	2 int16 LE	0.1A	-500 to 500A	raw=Ix10
SOC	1 uint8	0.5%	0-100%	raw=SOCx2
BMS Mode	1 uint8	-	0=Idle,1=Chg,2=Dischg,3=Fault	-
Contactor	1 uint8	-	0=Open,1=Closed	-
Fan Duty	1 uint8	1%	0-100%	direct
Fault Flags	1 bitfield	-	bit0 OverV,1 UnderV,2 OverT,3 OverI,4 Timeout	-

Physical plausibility ranges used for validation: Cell voltage 2.5-4.3V • Temperature -20 to 60C • Current -300 to 300A

3. Design Decisions & Fault Handling Strategy

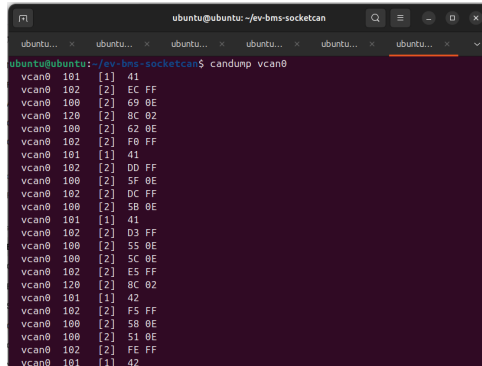
python-can + SocketCAN was chosen for cross-platform readability and mature can-utils interoperability. Coulomb counting is used for SOC — simplicity appropriate to a communication-network assignment. Dual-layer fault detection: the Sensor ECU self-reports faults (0x1F0) for fast Controller reaction, while the Diagnostic ECU independently re-validates every received value —

a malfunctioning Sensor that stops self-checking is still caught. **Event-triggered vs periodic split** follows real BMS practice: telemetry is periodic, actuator commands/faults are event-triggered to reduce bus load. **Timeout multiplier = 3x** period is a conservative choice, confirmed fast enough ($\leq 0.6\text{s}$) in testing below.

Trigger	Detection point	System response
Sensor value out of range	Sensor self-check + Diagnostic cross-check	0x1F0 -> Controller FAULT -> contactor OPEN, fan 100%
Periodic message missing >3x period	Diagnostic watchdog	0x1FF broadcast + diagnostic.log WARNING
ECU process killed	Diagnostic watchdog (missing periodic msgs)	Same as above; downstream nodes hold last state
Event-triggered node lost (e.g. Actuator)	Not timeout-monitored (by design)	Bus traffic continues; physical actuation gap only
Fault clears / ECU restarts	Diagnostic (message resumes)	"RESTORED" log entry; Controller returns to normal

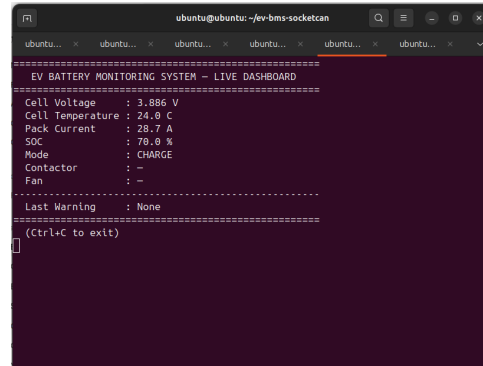
4. Demonstration — Challenge 1: Normal Operation

All ECUs started under normal conditions; periodic telemetry visible on candump, HMI showing live values with no faults.



```
ubuntu@ubuntu: ~/ev-bms-socketcan
ubuntu@ubuntu:~/ev-bms-socketcan$ candump vcan0
vcan0 101 [1] 41
vcan0 102 [2] EC FF
vcan0 100 [2] 69 0E
vcan0 120 [2] 8C 02
vcan0 100 [2] 62 0E
vcan0 102 [2] F8 FF
vcan0 101 [1] 41
vcan0 102 [2] DD FF
vcan0 100 [2] 5F 0E
vcan0 102 [2] DC FF
vcan0 100 [2] 5B 0E
vcan0 101 [1] 41
vcan0 102 [2] D3 FF
vcan0 100 [2] 55 0E
vcan0 100 [2] 5C 0E
vcan0 102 [2] E5 FF
vcan0 120 [2] 8C 02
vcan0 101 [1] 42
vcan0 102 [2] F5 FF
vcan0 100 [2] 5B 0E
vcan0 100 [2] 51 0E
vcan0 102 [2] FE FF
vcan0 101 [1] 42
```

Fig 4a — candump vcan0: steady periodic traffic on 0x100/0x101/0x102/0x120.

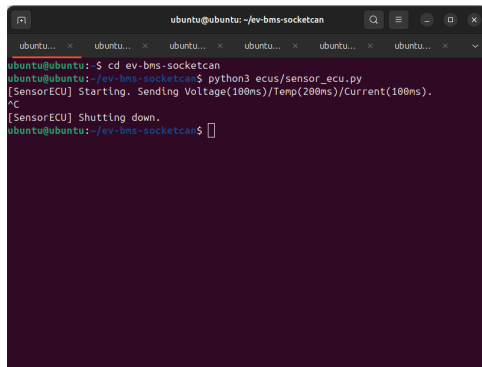


```
ubuntu@ubuntu: ~/ev-bms-socketcan
=====
EV BATTERY MONITORING SYSTEM - LIVE DASHBOARD
=====
Cell Voltage      : 3.886 V
Cell Temperature  : 24.0 C
Pack Current      : 28.7 A
SOC               : 70.0 %
Mode              : CHARGE
Contactor         : -
Fan               : -
=====
Last Warning      : None
=====
(Ctrl+C to exit)
```

Fig 4b — HMI: Voltage=3.886V, Temp=24.0C, Current=28.7A, SOC=70.0%, Mode=CHARGE, no warnings.

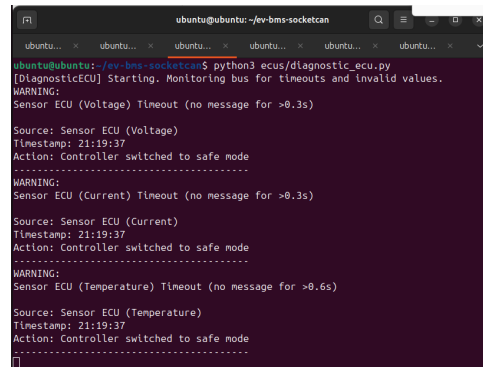
5. Demonstration — Challenge 2: Missing Message Detection

Sensor ECU terminated (Ctrl+C). Diagnostic watchdog detected the absence within the 3x-period window and logged timeouts for each affected signal.



```
ubuntu@ubuntu: ~/ev-bms-socketcan
ubuntu@ubuntu:~/ev-bms-socketcan$ cd ev-bms-socketcan
ubuntu@ubuntu:~/ev-bms-socketcan$ python3 ecus/sensor_ecu.py
[SensorECU] Starting. Sending Voltage(100ms)/Temp(200ms)/Current(100ms).
^C
[SensorECU] Shutting down.
ubuntu@ubuntu:~/ev-bms-socketcan$
```

Fig 5a — Sensor ECU terminated ("Shutting down.").



```
ubuntu@ubuntu: ~/ev-bms-socketcan
ubuntu@ubuntu:~/ev-bms-socketcan$ python3 ecus/diagnostic_ecu.py
[DiagnosticECU] Starting. Monitoring bus for timeouts and invalid values.
WARNING:
Sensor ECU (Voltage) Timeout (no message for >0.3s)

Source: Sensor ECU (Voltage)
Timestamp: 21:19:37
Action: Controller switched to safe mode
=====
WARNING:
Sensor ECU (Current) Timeout (no message for >0.3s)

Source: Sensor ECU (Current)
Timestamp: 21:19:37
Action: Controller switched to safe mode
=====
WARNING:
Sensor ECU (Temperature) Timeout (no message for >0.6s)

Source: Sensor ECU (Temperature)
Timestamp: 21:19:37
Action: Controller switched to safe mode
=====
```

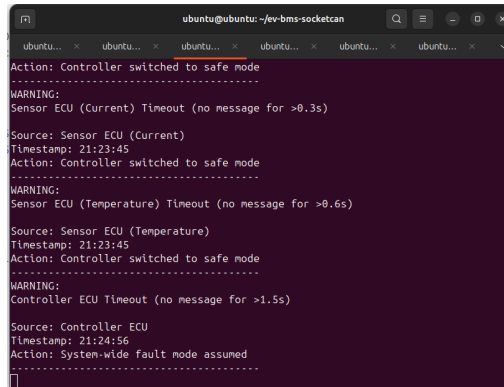
Fig 5b — Diagnostic ECU: Timeout WARNINGS for Voltage/Current (>0.3s) and Temperature (>0.6s) at 21:19:37, action "Controller switched to safe mode".

6. Demonstration — Challenge 3: Sensor Fault Injection

6.1 Over-temperature fault — Sensor ECU restarted with `--inject-temp 200`. Full fault chain triggered: self-check -> 0x1F0 -> Controller FAULT -> contactor open -> fan 100%.

7. Demonstration — Challenge 4: Node Failure

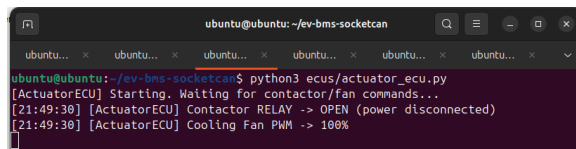
7.1 Controller ECU killed — 0x120 stopped; Diagnostic watchdog detected the missing node and logged a system-wide fault.



```
ubuntu@ubuntu: ~/ev-bms-socketcan
Action: Controller switched to safe mode
-----
WARNING:
Sensor ECU (Current) Timeout (no message for >0.3s)
Source: Sensor ECU (Current)
Timestamp: 21:23:45
Action: Controller switched to safe mode
-----
WARNING:
Sensor ECU (Temperature) Timeout (no message for >0.6s)
Source: Sensor ECU (Temperature)
Timestamp: 21:23:45
Action: Controller switched to safe mode
-----
WARNING:
Controller ECU Timeout (no message for >1.5s)
Source: Controller ECU
Timestamp: 21:24:56
Action: System-wide fault mode assumed
-----
```

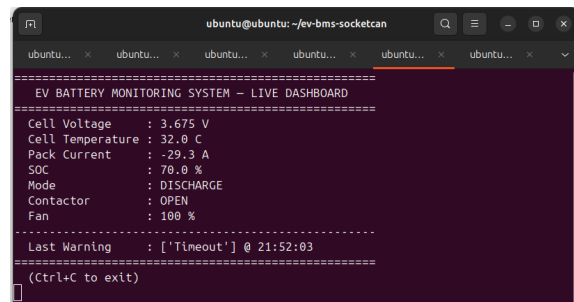
Fig 7a — Diagnostic ECU: "Controller ECU Timeout (no message for >1.5s)" at 21:24:56, action "System-wide fault mode assumed".

7.2 Actuator ECU killed — with Sensor/Controller/Diagnostic/HMI still running, the Actuator was terminated. Since 0x110/0x111 are event-triggered (not periodic), Diagnostic correctly does not flag this as a timeout — it is a physical-actuation gap by design, not a monitored bus fault. No other ECU crashed.



```
ubuntu@ubuntu: ~/ev-bms-socketcan
python3 ecus/actuator_ecu.py
[ActuatorECU] Starting. Waiting for contactor/fan commands...
[21:49:30] [ActuatorECU] Contactor RELAY -> OPEN (power disconnected)
[21:49:30] [ActuatorECU] Cooling Fan PWM -> 100%
```

Fig 7b — Actuator ECU terminated cleanly (Ctrl+C); the SocketcanBus notice is a harmless python-can cleanup message.

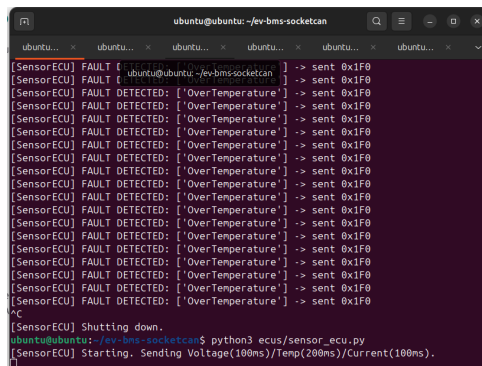


```
ubuntu@ubuntu: ~/ev-bms-socketcan
EV BATTERY MONITORING SYSTEM - LIVE DASHBOARD
-----
Cell Voltage      : 3.675 V
Cell Temperature  : 32.0 C
Pack Current      : -29.3 A
SOC               : 70.0 %
Mode              : DISCHARGE
Contactor         : OPEN
Fan               : 100 %
-----
Last Warning      : ['Timeout'] @ 21:52:03
-----
(Ctrl+C to exit)
```

Fig 7c — HMI continued running normally post-kill; last commanded Contactor/Fan state remained visible with no crash elsewhere.

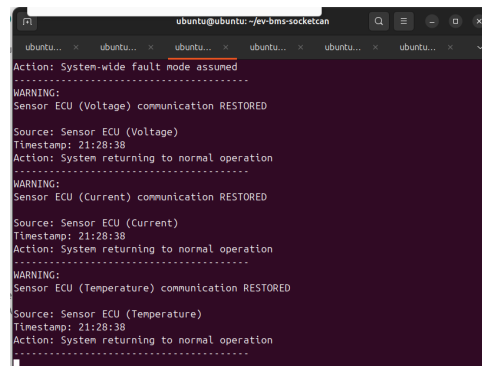
8. Demonstration — Challenge 5: Recovery

Sensor ECU restarted (no fault flag). Diagnostic watchdog detected resumed periodic messages and logged recovery for each.



```
ubuntu@ubuntu: ~/ev-bms-socketcan
[SensorECU] FAULT [ ubuntu@ubuntu: ~/ev-bms-socketcan ] -> sent 0x1F0
[SensorECU] FAULT [ ubuntu@ubuntu: ~/ev-bms-socketcan ] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] FAULT DETECTED: ['OverTemperature'] -> sent 0x1F0
[SensorECU] Shutting down.
ubuntu@ubuntu: ~/ev-bms-socketcan$ python3 ecus/sensor_ecu.py
[SensorECU] Starting. Sending Voltage(100ms)/Temp(200ms)/Current(100ms).
```

Fig 8a — Sensor ECU restarted cleanly, resuming normal periodic transmission.



```
ubuntu@ubuntu: ~/ev-bms-socketcan
Action: System-wide fault mode assumed
-----
WARNING:
Sensor ECU (Voltage) communication RESTORED
Source: Sensor ECU (Voltage)
Timestamp: 21:28:38
Action: System returning to normal operation
-----
WARNING:
Sensor ECU (Current) communication RESTORED
Source: Sensor ECU (Current)
Timestamp: 21:28:38
Action: System returning to normal operation
-----
WARNING:
Sensor ECU (Temperature) communication RESTORED
Source: Sensor ECU (Temperature)
Timestamp: 21:28:38
Action: System returning to normal operation
-----
```

Fig 8b — Diagnostic ECU: Voltage/Current/Temperature "communication RESTORED" at 21:28:38, action "System returning to normal operation".

9. Test Case Results

ID	Test	Result
F1-F7	Voltage/Temp/Current/Status TX-RX, Contactor/Fan commanding, HMI live update	Pass — all 7 functional tests confirmed via candump + dashboard
T1	Missing message (Challenge 2)	Pass — timeouts logged for all 3 signals within spec
T2	Invalid sensor value — temperature (Challenge 3)	Pass — full fault chain executed correctly
T3	Invalid sensor value — voltage	Pass — OverVoltage fault chain confirmed identically to T2
T4	Node failure — Controller (Challenge 4)	Pass — timeout detected, safe-mode assumed
T5	Node failure — Actuator	Pass — no crash; event-triggered loss correctly out of timeout scope
T6	Recovery (Challenge 5)	Pass — all 3 RESTORED events logged correctly
I1	End-to-end normal run	Pass — continuous run, no false faults
I2	Combined fault scenario (Diagnostic ECU also killed)	Not Run — recommended future work
I3	Full challenge sequence	Pass — clean state progression through all challenges

Summary: 11 of 12 planned test cases were executed and passed. Only I2 (Diagnostic ECU itself killed during a fault) remains untested — a low-priority edge case since Diagnostic ECU is a passive observer and its absence does not affect the Sensor->Controller->Actuator fault-response chain, which is independently confirmed by T2/T3. All 5 required verification challenges, plus both extended fault variants (over-voltage, Actuator node failure), were demonstrated successfully with terminal, dashboard, and log evidence.

10. Conclusion & Known Limitations

The implemented system successfully demonstrates a 4-ECU distributed automotive network over SocketCAN, covering periodic and event-triggered messaging, a layered fault-detection strategy (local self-check + independent bus-level validation), and safe-state actuation on fault. All five required verification challenges — plus two extended variants (over-voltage injection and Actuator node failure) — were demonstrated live with terminal, dashboard, and log evidence.

Known limitations:

- SOC model is simplistic coulomb counting with no temperature compensation or cell balancing.
- Single simulated cell rather than a full pack; the same message pattern scales directly to multi-cell designs via a cell-index byte.
- No bus-off / error-frame handling implemented; SocketCAN's default handling is relied upon.
- The I2 combined-fault integration test (Diagnostic ECU killed simultaneously with a sensor fault) was not exercised in the demo session.